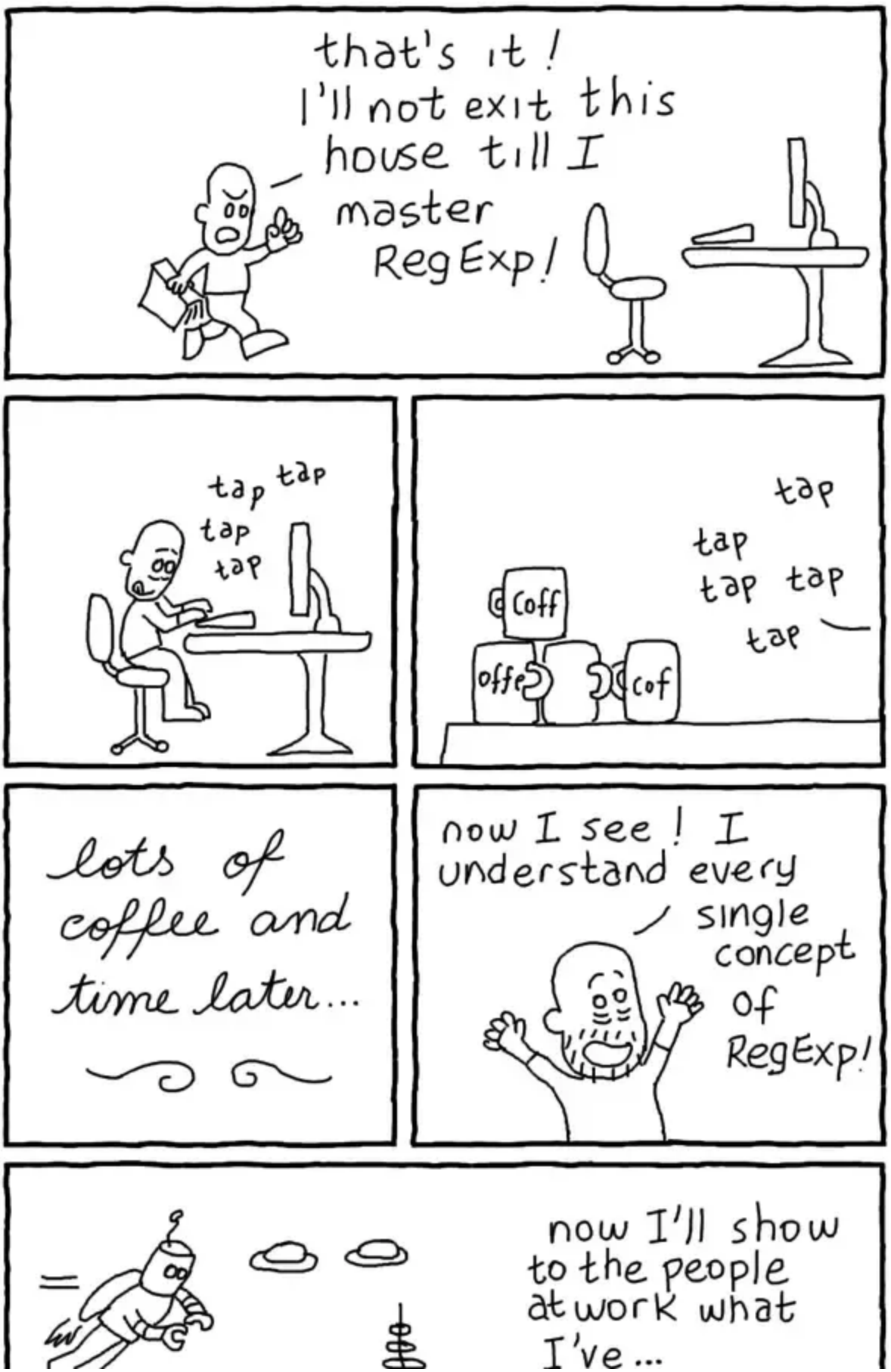
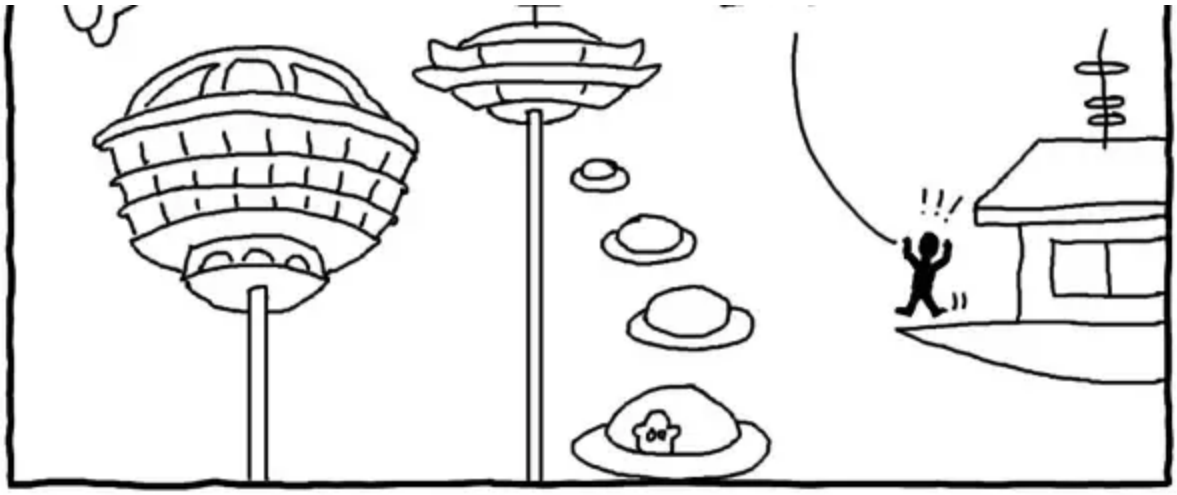
# Regex

Regular Expressions


that's it! I'll not exit this house till I master RegExp!
tap tap tap tap tap
tap tap tap tap tap
Coff
offe
cof
lots of coffee and time later...
now I see! I understand every single concept of RegExp!
now I'll show to the people at work what I've...



Let's start with a simple example where we are trying to find the word 'cool' in the text.

```
Batman is cool
```

With regex, we could simply type out the word 'cool' as the pattern and it will match the word.

```
'cool'
```

While regex matched our desired word '**cool**', the way it operates is not at the word level but the character level. This is the key idea.

> ***Key Idea**: Regex works at the character-level, not word-level.*




The implication of this is that the regex `r'cool'` would match the following sentences as well.

```
Batman is coolest

He bought a watercooler

Batman is supercool
```

## Basic building blocks and Rules

#### Specific character

We can simply specify the character in the regular expression and it will match all instances in the text.

For example, a regular expression given below will match all instances of 'a' in the text. You can use any of the small and capital alphabets.

```
'a'
```



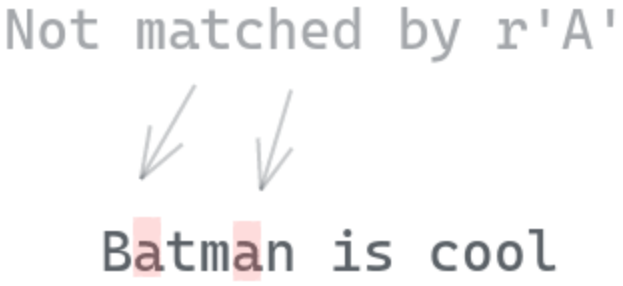

You can also use any digits from 0 to 9 and it will work as well.

```
'3'
```




Note that regex is case-sensitive by default and thus the following regex won't match anything.

```
'A'
```




**White space characters [Space, Tab, Newline]**

We can detect special characters such as whitespace and newlines using special escape sequences.




Besides the common ones above, we have:

- **\r** for carriage return
- **\f** for form feed
- **\e** for escape.

## Special sequences

> *Regex provides a bunch of built-in special symbols that can match a group of characters at once.* ***These begin with backslash*** `\`

- **Pattern:** `\d`

  It matches any single-digit number between 0 to 9.

  ```
  I have 2 laptops and 1 smartphone
  ```

  Notice that matches are single digit. So we have 4 different matches below instead of a single number `18.04`.




- **Pattern: \s**

  It matches any whitespace character (space, tab or newline).



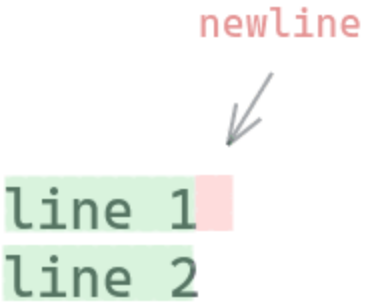

- **Pattern: \w**

  It matches any of the small alphabets(a to z), capital alphabets(A to Z), digits (0 to 9), and underscore.




- **Pattern: .**

  It matches any character except the new line (\n).




```
import re

>>> re.findall(r'.', 'line 1\nline2')
['l', 'i', 'n', 'e', ' ', '1', 'l', 'i', 'n', 'e', '2']
```

- **Pattern: Negations**

  If you use the capitalised versions of the patterns above, they act as negation.

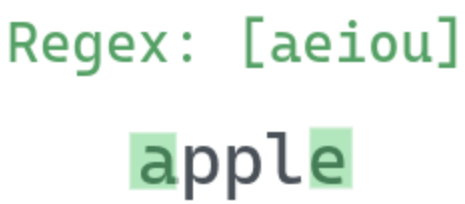
  For example, if "\d" matched any digits from 0 to 9, then "\D" will match anything except "0 to 9".




## Character sets

These are patterns starting with `[` and ending with `]` and specify the characters that should be matched enclosed by brackets.

For example, the following pattern matches any of the characters 'a', 'e', 'i', 'o', and 'u'.



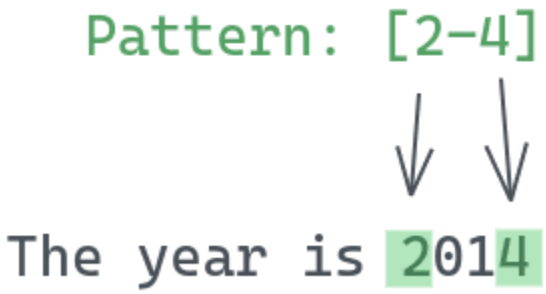

You can also replicate the functionality of `\d` using the below pattern. It will match any digits between 0 to 9.



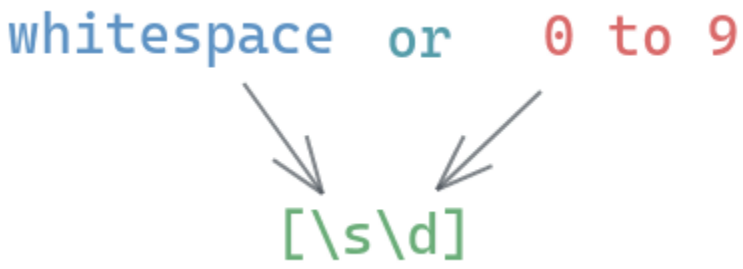

Instead of specifying all the digits, we can use `-` to specify only start and end digits. So, instead of `[0123456789]`, we can do:




For example, `[2-4]` can be used to match any digits between 2 to 4 i.e. (2 or 3 or 4).




You can even use the special characters we learned previously inside the brackets. For example, you can match any digit from 0 to 9 or white-space as:




*Below, I have listed some useful common patterns and what they mean.*

Any small alphabet between a to z

[a-z]

Any capital alphabet between A to Z

[A-Z]

Any small or capital alphabet

[a-zA-Z]

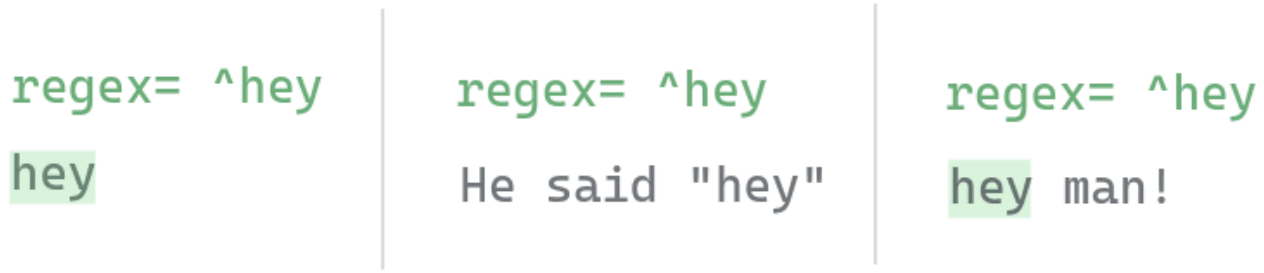
Any small or capital alphabet or numeric digit

[a-zA-Z0-9]

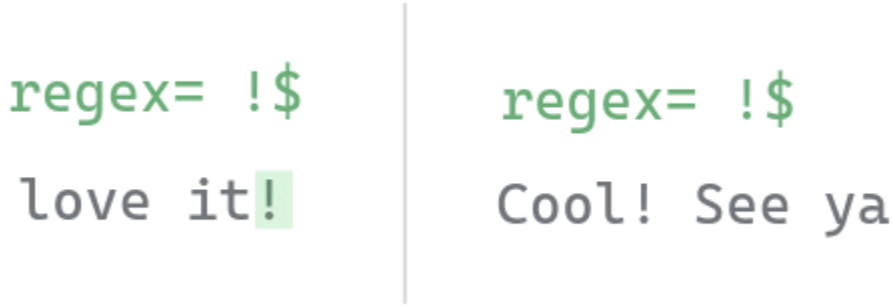
## Anchors

Regex also has special handlers to make the pattern only match if it's at the start or end of the string.

> *We can use the `^` anchor to match patterns only at the start of a line. For example:*



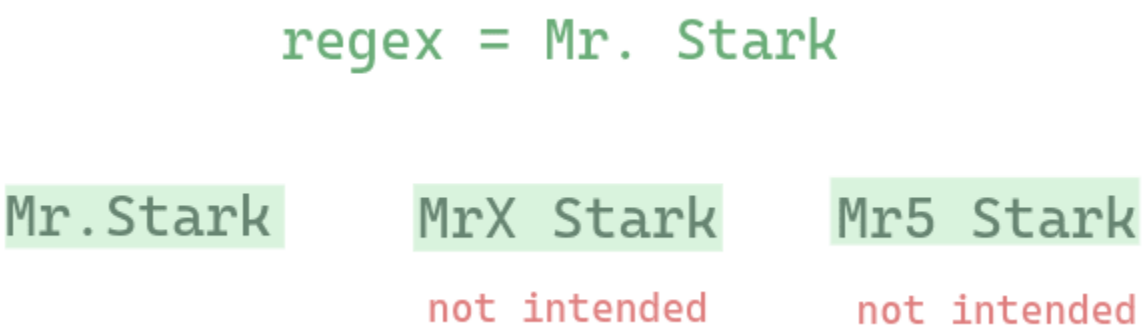

> *Similarly, we can use the `$` anchor after the character to match patterns only if it's the end of the line. For example:*



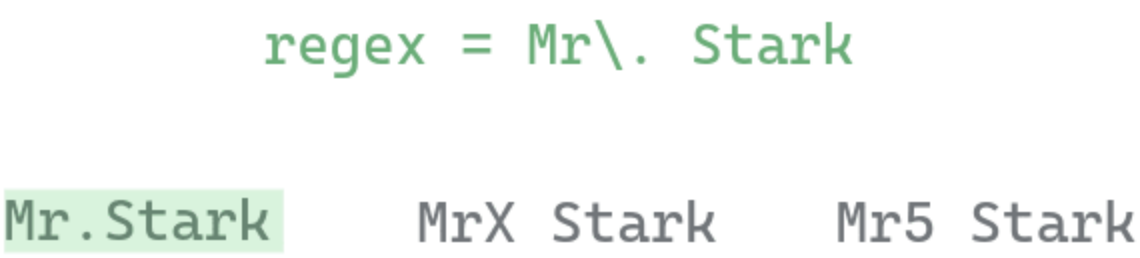

## Escaping metacharacters

Consider a case where we want to exactly match the word "Mr. Stark".

If we write a regex like `Mr. Stark`, then it will have an unintended effect. Since we know dot has a special meaning in a regex.



So, we should always escape the special metacharacters like `.`, `$` etc. if our goal is to match the exact character itself.




Here is the list of metacharacters that you should remember to escape if you're using them directly.

```
^ $ . * + ? { } [ ] \ | ( )
```

## Quantifiers

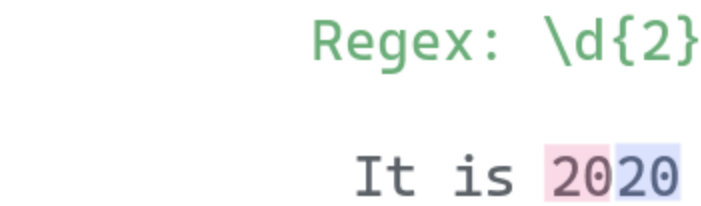
Regex provides special quantifiers to specify different types of repetition for the character preceding it.

**Fixed repetition**

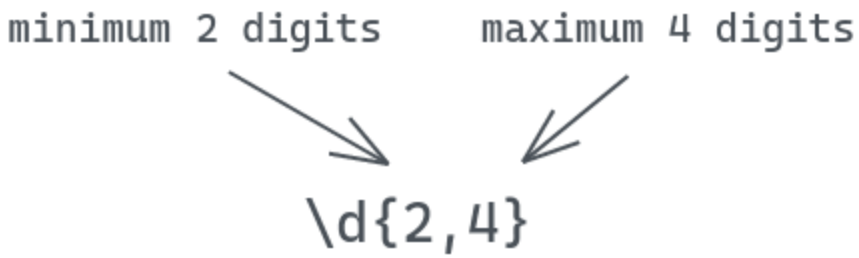
We can use the `{...}` quantifier to specify the number of times a pattern should repeat.




For example, the previous pattern for matching 2-digit number can be recreated as:




You can also specify a range of repetitions using the same quantifier. For example, to match from 2-digit to 4-digit numbers, we could use the pattern:




When applied to a sentence, it will match both 4-digit and 2-digit numbers.

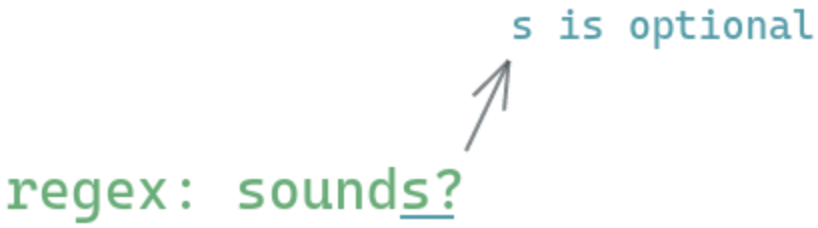



> ***Note:***
> *There should not be any space between minimum and maximum count For example, \d{2, 4} doesn't work.*

**Flexible quantifiers**
Regex also provides quantifiers "*", "+" and "?" using which you can specify flexible repetition of a character.

- **0 or 1 times:** `?`

  The `?` quantifier matches the previous character if it repeats 0 or 1 times. This can be useful to make certain parts optional.




For example, let's say we want to match both the word "sound" and "sound" where "s" is optional. Then, we can use the `?` quantifier that matches if a character repeats 0 or 1 times.

```
regex: sounds?

It sounds cool
They made it sound cool
```

- **one or more times:** `+`

  The `+` quantifier matches the previous character if it repeats 1 or more times. For example, we could find numbers of any arbitrary length using the regex `\d+`.

```
regex= \d+

My code is 1
My code is 12
My code is 123
My code in 12345
```

- **zero or more times:** `*`

  The `*` quantifier matches the previous character if it repeats zero or more times.

That is enough basic to get you started!

You should try building and testing some Regex here now

https://regexr.com/

RegExr: Learn, Build, & Test RegEx • regexr.com

If you want to get some extra knowledge check these,

## References


1. https://medium.com/factory-mind/regex-tutorial-a-simple-cheatsheet-by-examples-649dc1c3f285
2. https://github.com/ziishaned/learn-regex
3. https://gist.github.com/tapanhp/c57cea548c51cd6372b548ff0ae7b551


**Credits**

> *Source for this blog*